

How GPU Texture Filters Work: Nearest Neighbor Interpolation vs. Bilinear Filtering

Group 7 Members:

Arugay, Enrico Joaquin C.

Cailao, Carlos Luis B.

Felix, Catherine Liberty B.

Gamboa, Raphael R.

Gutierrez, Michael Luis J.

CSARCH2 - S40

Date of Submission:

06/17/2026

I. Chosen Topic

A. Category: How It Works

B. Theme: GPU Texture Filtering

C. Title: How GPU Filter Textures Work: Nearest Neighbor Interpolation vs. Bilinear Filtering

II. Introduction

When a coordinate does not lie on texel centers, where defined color values lie, the Graphics Processing Unit (GPU) must find a color for that specific coordinate. This often happens when 2D textures are stretched over 3D objects. To find that color, the GPU uses texture filtering. This Virtual Exhibit demonstrates how the Texture Mapping Unit (TMU), a GPU hardware, calculates for the closest texels and fetches the colors of those texels from the GPU's L1 Cache all the way down to the Video RAM (VRAM). To present this process, the group will be focusing on two fundamental filtering techniques, Nearest Neighbor Filtering and Bilinear Filtering.

A. Key Hardware Components

1. Arithmetic Logic Units

- a) Processing units in the GPU used primarily for processing visuals and executing related calculations

2. Texture Mapping Unit (TMU)

- a) Found in the GPU
- b) This is the unit responsible for sampling textures and filtering
- c) In the process of filtering, it fetches texels and outputs the filtered color value

3. L1 Cache

- a) Local high-speed cache located in each compute unit
- b) Designed with a 2D layout in consideration of the spatial layout of textures

4. L2 Cache

- a) Shared GPU cache

5. Video RAM (VRAM)

- a) Built-in GPU memory

- b) Much faster than RAM
- c) Holds raw texture data (texels)

B. Nearest Neighbor Interpolation

1. The simplest and fastest method, albeit rarely used
 - a) The TMU receives a texture coordinate (u, v)
 - b) It then calculates the coordinates of the texel closest to the received sample point using rounding
 - c) It takes the color of the single closest pixel and returns that exact texel
2. Inefficient since it could lead to grainy or blocky results

C. Bilinear Filtering

1. More accurate and more common than nearest neighbor filtering (implemented in modern graphics hardware)
 - a) The TMU simultaneously finds the 4 closest texels to the sample point
 - b) Blends these 4 texels, resulting in a custom color
2. Might result in blurry or muddy textures

III. Detailed Plan

A. Tech Stack

1. Runtime Environment: Node.js
2. Core Framework: Astro 6
3. Component Architecture: React 19 & MDX
4. Styling Framework: Tailwind CSS

B. Interactive Elements

1. Magnifier Slider
 - a) A dynamic slider that allows users to adjust magnification factor scaling from 1x to 32x. The step increment of the coordinate sampler array will be shown in real-time
2. Live Console
 - a) A display box alongside the canvas that shows the exact calculations happening behind the scenes.

3. Toggle
- a) A switch that changes the sampling function between Nearest Neighbor and Bilinear.

C. Style Guide

1. Sample Layout

 Computer Architecture Virtual Exhibit

S40 - Texture Filtering in 3D Graphics

 How GPU Filter Textures: Nearest Neighbor vs Bilinear Filtering

Introduction

When a GPU renders a 3D scene, it needs to apply textures to objects. But what happens when a texture is viewed at different distances or angles? The GPU uses **texture filtering** to determine pixel colors — and two of the most fundamental methods are **Nearest Neighbor** and **Bilinear Filtering**.

What is Texture Filtering?

Texture filtering is the process of **sampling a texture image** and calculating the correct color for each pixel on screen. It's a critical GPU function that affects:

- **Visual Quality** — sharp or blurry?
- **Performance** — fast or slow?
- **Memory Usage** — how much data is needed?

The Problem: Pixels vs Texels

Term	Definition
Pixel	A single dot on your screen
Texel	A single dot in a texture image
The Problem 1 pixel doesn't always match 1 texel (especially when zooming)	

When a texture is displayed larger or smaller than its original size, the GPU must **interpolate** (calculate) the missing pixels.

Nearest Neighbor Interpolation

How it works: The GPU simply picks the closest texel to the sample point. No averaging, no blending.

Formula: $P(x,y) = P(\text{round}(x), \text{round}(y))$

Advantages:

- Fastest filtering method
- Simple to implement
- Preserves sharp edges

Disadvantages:

- Blocky, pixelated appearance when zoomed in
- Noticeable aliasing (jagged edges)
- Poor quality for realistic images

Where it's used:

- Pixel art games (like Minecraft)
- Reno-style games
- Older hardware

Bilinear Filtering

How it works: The GPU averages the 4 nearest texels (a 2x2 grid) and blends them together.

Formula: $P(x,y) = (TL + TR + BL + BR) / 4$

Advantages:

- Smooth appearance
- No obvious pixelation
- Better visual quality

Disadvantages:

- Slower than Nearest Neighbor
- Can appear slightly blurry
- Uses more GPU resources

Where it's used:

- Most modern games
- 3D applications
- Textures viewed at normal distances

Interactive Demo

Click the toggle to switch between filtering modes. Use the slider to zoom in and see the difference!

Texture Explorer

Compare Nearest Neighbor vs Bilinear Filtering

1x

1x

32x

Compare:

Nearest Neighbor

 Uses closest pixel only — fast but blocky



zoom 1x

mode Nearest Neighbor

formula $P(x,y) = P(\text{round}(x), \text{round}(y))$

performance

Fastest (1x)

Activity Log

Zoom: 7x - Notice the difference in edges

Zoom: 6.5x - Notice the difference in edges

Zoom: 6x - Notice the difference in edges

Zoom: 5.5x - Notice the difference in edges

Zoom: 5x - Notice the difference in edges

Zoom: 4.5x - Notice the difference in edges

Zoom: 4x - Notice the difference in edges

Zoom: 1x - Compare the two methods!

Type a command... (help, toggle, compare, formula) Enter

Nearest Neighbor vs Bilinear

Nearest Neighbor

ACTIVE

Uses closest pixel only — fast but blocky

$P(x,y) = P(\text{round}(x), \text{round}(y))$

Fastest (1x)

Bilinear

Averages 2x2 surrounding pixels — smooth but blurry

$P(x,y) = (TL + TR + BL + BR) / 4$

Good (2x)

Tip: Click the toggle to switch between Nearest Neighbor and Bilinear! Zoom in to see the difference clearly. Type help in the console for commands. Try typing compare for more info!

Comparison Table

Feature	Nearest Neighbor	Bilinear
Quality	Low (blocky)	Medium (smooth)
Performance	Fastest	Good
GPU Usage	Very Low	Low
Memory Usage	None	None
Formula	Closest texel	$(TL + TR + BL + BR) / 4$

When to Use Each Method

Scenario	Recommended Method	Why
Pixel art games	Nearest Neighbor	Preserves blocky aesthetic
Retro style games	Nearest Neighbor	Authentic retro look
Modern 3D games	Bilinear	Good balance of quality/speed
VR/AR applications	Anisotropic	Best quality for angles
Mobile games	Bilinear	Performance-focused

How This Works in the GPU Pipeline

- 1. **Texture Loading** — GPU loads texture into memory
- 2. **UV Mapping** — Texture coordinates are calculated
- 3. **Sample Point** — GPU finds where to sample on texture
- 4. **Filtering Applied** — Nearest or Bilinear (or other)
- 5. **Pixel Output** — Final color is displayed on screen

Fun Facts

- **Nearest Neighbor** was the default in early 3D games (like PlayStation 1)
- **Bilinear** became popular with the PlayStation 2 era
- Modern GPUs can process **100+ billion texels per second**
- **Anisotropic filtering** (the next step up) was introduced in 1999 with NVIDIA GeForce 256

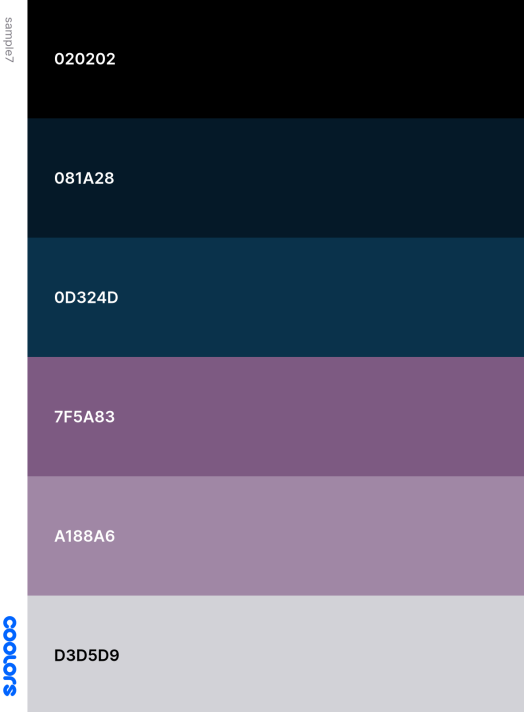
Learn More

- [GPU Texture Filtering Explained \(NVIDIA\)](#)
- [Wikipedia: Texture Filtering](#)
- [Bilinear vs Nearest Neighbor Comparison](#)

Computer Architecture is Forever | Virtual Museum Exhibit 2026

Computer Architecture is Forever | Virtual Museum Exhibit 2026

2. Palette



Computer Architecture Virtual Exhibit

How GPU Filters Textures: Nearest Neighbor vs Bilinear Filtering

Introduction

When a GPU renders a 3D scene, it needs to apply textures to objects. But what happens when a texture is viewed at different distances or angles? The GPU uses texture filtering to determine pixel colors — and two of the most fundamental methods are **Nearest Neighbor** and **Bilinear Filtering**.

What is Texture Filtering?

When a GPU renders a 3D scene, it needs to apply textures to objects. But what happens when a texture is viewed at different distances or angles? The GPU uses texture filtering to determine pixel colors — and two of the most fundamental methods are **Nearest Neighbor** and **Bilinear Filtering**.

Pixels vs. Texels

When a GPU renders a 3D scene, it needs to apply textures to objects. But what happens when a texture is viewed at different distances or angles? The GPU uses texture filtering to determine pixel colors — and two of the most fundamental methods are **Nearest Neighbor** and **Bilinear Filtering**.

Nearest Neighbor Filtering

When a GPU renders a 3D scene, it needs to apply textures to objects. But what happens when a texture is viewed at different distances or angles? The GPU uses texture filtering to determine pixel colors — and two of the most fundamental methods are **Nearest Neighbor** and **Bilinear Filtering**.

Advantages

When a GPU renders a 3D scene, it needs to apply textures to objects. But what happens when a texture is viewed at different distances or angles? The GPU uses texture filtering to determine pixel colors — and two of the most fundamental methods are **Nearest Neighbor** and **Bilinear Filtering**.

Disadvantages

When a GPU renders a 3D scene, it needs to apply textures to objects. But what happens when a texture is viewed at different distances or angles? The GPU uses texture filtering to determine pixel colors — and two of the most fundamental methods are **Nearest Neighbor** and **Bilinear Filtering**.

Bilinear Filtering

When a GPU renders a 3D scene, it needs to apply textures to objects. But what happens when a texture is viewed at different distances or angles? The GPU uses texture filtering to determine pixel colors — and two of the most fundamental methods are **Nearest Neighbor** and **Bilinear Filtering**.

Interactive Demo

Filtering Mode

Primary Background: #020202

Secondary Background: #081a28

Accent Colors: #0d324d, #7f5a83, #a188a6

Font Color: #d3d5d9

3. Typography

- a) Headings: Georgia, serif
- b) Body: Georgia, serif
- c) Console: Courier New, monospace
- d) Interactive Labels: System-UI stack
- e) Footer: System-UI stack

D. Target Repository Structure

csarch2-case-project

├── astro.config.mjs

├── package.json

├── tsconfig.json

└── src/

 ├── components/

 └── TextureSimulator.jsx # Houses unified state logic & LERP math

blocks

├── data/

└── exhibit-info.json # Contains team index, title metadata, and

category parameters

├── layouts/

└── ExhibitLayout.astro # Frame layout injector containing layout

templates

├── pages/

└── index.mdx #Main exhibit content

simulator component

└── styles/

 └── global.css # System directives for Tailwind utility injection

IV. References

Documentation - Texture filtering. (n.d.). Arm Developer.

<https://developer.arm.com/documentation/102449/0200/Texture-filtering>

Stevewhims. (n.d.-a). *Bilinear texture filtering - UWP applications*. Microsoft Learn.

<https://learn.microsoft.com/en-us/windows/uwp/graphics-concepts/bilinear-texture-filtering>

Stevewhims. (n.d.-a). *Nearest-point sampling - UWP applications*. Microsoft Learn.

<https://learn.microsoft.com/en-us/windows/uwp/graphics-concepts/nearest-point-sampling>

Stevewhims. (n.d.). *Texture filtering - UWP applications*. Microsoft Learn.

<https://learn.microsoft.com/en-us/windows/uwp/graphics-concepts/texture-filtering>

Wikipedia contributors. (2026, March 4). Texture filtering. Wikipedia.

https://en.wikipedia.org/wiki/Texture_filtering